

Presidency

School of Artificial Intelligence and Advanced Computing

CSS7102- Mini Project Review-4

EngageIQ - Responsible AI Framework for Remote Employee Engagement Analytics

Batch Number: CSD_23

Roll Number	Student Name
20231CSD0090	M Sania Mehek
20231CSD0088	Irfan Basha L
20231CSD0105	Madiha
20231CSD0134	Akash M S

Under the Supervision of,

Mr. Sahabzada Betab Badar
Assistant Professor
School of Artificial Intelligence and Advanced Computing
Presidency University

Name of the Program: B.Tech Computer Science Engineering (DATA SCIENCE)
Name of the HoD: Dr. S Pravinth Raja
Name of the Program Project Coordinator: Dr. Sameena Banu
Name of the School Project Coordinators: Dr. Sampath A K

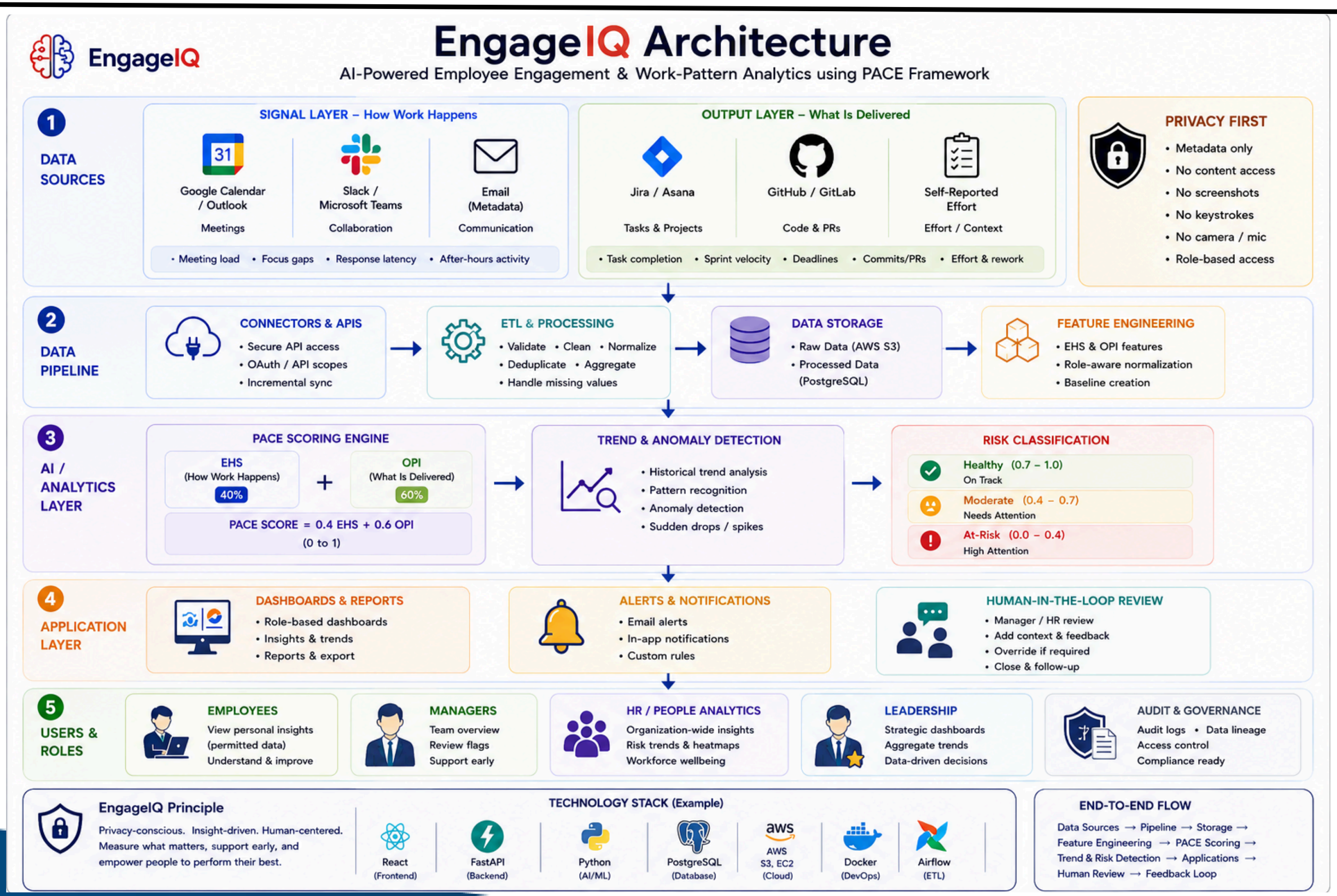
Content

- ABSTRACT
- ARCHITECTURE DIAGRAM & MODULES
- OBJECTIVES
- PROGRESS STATUS

ABSTRACT

- EngageIQ is a responsible AI-powered framework designed to analyze employee engagement and work patterns in remote and hybrid workplaces without invasive surveillance.
- It uses a dual-lens approach: the **Signal Layer** analyzes metadata such as meeting load, focus-time gaps, response latency, and after-hours activity, while the **Output Layer** evaluates tangible deliverables such as task completion, sprint velocity, deadlines, commits, and pull requests.
- These are combined using the **PACE Score (40% EHS + 60% OPI)** to prioritize actual outcomes.
- Trend and anomaly detection identify changes over time, while risk classification highlights Healthy, Moderate, and At-Risk patterns. Role-based dashboards, alerts, privacy controls, and human-in-the-loop review ensure that insights support **responsible decision-making rather than automated judgment.**

ARCHITECTURE DIAGRAM & MODULES



OBJECTIVES

1. **Measure employee engagement patterns**
2. **Measure actual work outcomes**
3. **Develop the PACE scoring system**
4. **Provide role-aware risk classification**
5. **Enable responsible, human-in-the-loop decisions**
6. Detect trends and potential burnout risks
7. Protect employee privacy and build trust

PROGRESS FROM PREVIOUS REVIEW

Subsystem	Implemented in Codebase	Incomplete / Mocked (Target for Next Phases)
Frontend Dashboards	Full UI views: Overview, OrgView, TeamView, EmployeeView, PaceScore simulator. Connected to live backend (`mock.ts` replaced).	✓ All views live — nothing remaining.
Data Ingestion UI	Drag-and-drop CSV uploader calling `POST /api/upload` .	Lacks Live Sync trigger, provider status indicators, and mode switcher.
Integrations UI	None.	Need dedicated `/settings/integrations` view with configuration modals.
Backend Models	All models (Employee, PaceScore, Team, Role, SignalMetrics, OutputMetrics, RiskEvent, ReviewRecord, IntegrationConfig) implemented in `models.py` .	✓ All models implemented — nothing remaining.
REST Endpoints	All REST Endpoints implemented in `main.py` (upload, scores, organization, teams, employees, integrations).	✓ All endpoints implemented — nothing remaining.
Scoring Logic	Implemented pipeline to compute and map data to planned metrics for EHS, OPI, and PACE score analytics (in `pace_engine.py`).	Needs rolling window baselines and role matrices. Need to explore/understand ML models (optimization, fit) & address production edge cases, space & time complexity.
Anomaly Detection	Simple client-side heuristics in `src/lib/pace.ts` .	Needs backend Scikit-learn Isolation Forest and PyTorch LSTM engine.
Credential Security	None.	Needs Fernet symmetric encryption vault for external API keys.
External Connectors	Implemented third-party connectors (GitHub, Jira, Slack, Google Calendar).	✓ All four connectors complete — nothing remaining.